Introduction aux LLMs

Séance 5 : Frontières et raisonnement

ESGI Paris

Célia Nouri · `celia.nouri@inria.fr`

Semestre 1, 2026–2027

Agenda

1. Architectures agentiques : ReAct, planification, mémoire, systèmes multi-agents
2. Modèles de raisonnement : o1/o3, DeepSeek-R1, inference-time compute
3. (bonus) Multimodalité : vision-langage en bref
4. Préparation à l'examen & révisions

1. Architectures agentiques

Du function calling à l'agent

Séance 4 : un LLM peut émettre **un** appel d'outil, recevoir **un** résultat, puis répondre.

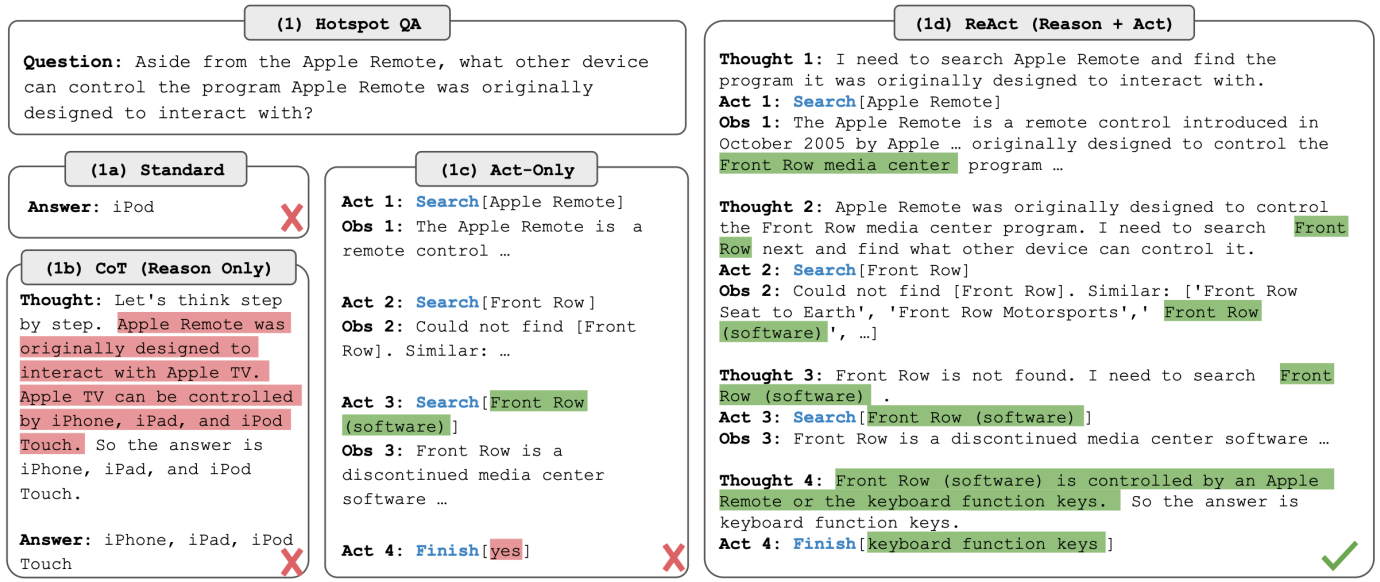
Un **agent** va plus loin : il **enchaîne** plusieurs cycles raisonnement → action → observation, de façon autonome, jusqu'à résoudre une tâche complète.

```
Function calling : 1 question → 1 appel d'outil → 1 réponse
Agent            : 1 objectif → N cycles (raisonner, agir, observer) → résultat
```

Trois briques à assembler : **agir en boucle** (ReAct), **planifier**, et **se souvenir** d'une exécution à l'autre.

ReAct : raisonner et agir en boucle

Yao et al. (2022/2023, Princeton/Google), *"ReAct: Synergizing Reasoning and Acting in Language Models"*.



ReAct : raisonner et agir en boucle

Combine le Chain-of-Thought (séance 4) avec des appels d'outils, **en boucle** :

```
Thought      : Il me faut la population actuelle de Paris pour répondre.
Action       : search("population Paris 2026")
Observation  : "Environ 2,1 millions d'habitants intra-muros."
Thought      : J'ai l'information nécessaire, je peux répondre.
Final Answer : Paris compte environ 2,1 millions d'habitants.
```

Chaque observation **enrichit le contexte** pour l'étape de raisonnement suivante — le modèle peut aussi corriger sa trajectoire si une action échoue ou renvoie un résultat inattendu.

ReAct : Finetuning SLM pour la génération de prompt

Les LLMs ayant déjà la capacité de raisonner et d'agir (séance 4), ReAct fonctionne via un LLM pré-entrainé, prompté avec des exemples (few-shot) démontrant la génération d'actions et de raisonnements en langage naturel pour répondre à des questions similaires au prompt.

Dans un deuxième temps, un plus petit LLM est finetuné (en utilisant les trajectoires ayant produit de réponses correctes) pour générer les trajectoires (raisonnements + actions + observations) conditionnées sur les questions ou tâches d'entrée. Ces trajectoires générées sont ensuite utilisées comme prompt pour le LLM non-finetuné.

Planifier avant d'agir : ReWOO

Xu et al. (2023) — *"ReWOO: Decoupling Reasoning from Observations"*.

Dans ReAct, chaque observation ré-injecte **tout le contexte** dans un nouvel appel au LLM → coûteux en tokens si beaucoup d'étapes.

ReWOO (Reasoning WithOut Observation) sépare planification et exécution :

```
1. Planner : génère TOUT le plan d'appels d'outils d'un coup (sans les exécuter)
2. Worker  : exécute chaque appel du plan, indépendamment
3. Solver  : rassemble les résultats et rédige la réponse finale
```

ReWOO

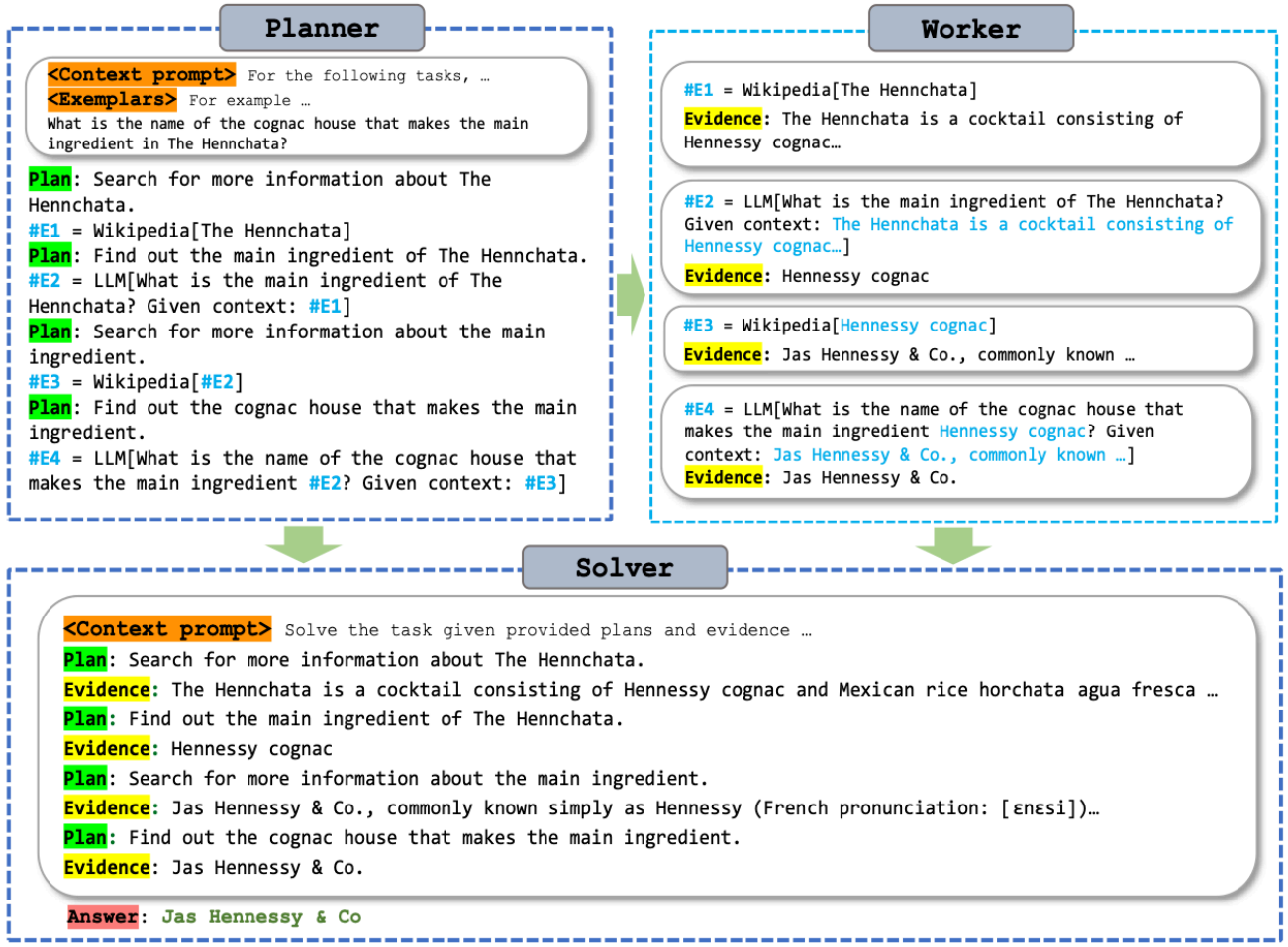


Figure 1: Workflow of ReWOO . Given a question, Planner composes a comprehensive blueprint of interlinked plans prior to tool response. The blueprint instructs Worker to use external tools and collect evidence. Finally, plans and evidence are paired and fed to Solver for the answer.

Comparaison ReAct et ReWOO

	ReAct	ReWOO
Appels au LLM	1 par cycle (Thought/Action/Observation)	1 pour planifier + 1 pour conclure
Adaptatif si un résultat surprend ?	Oui, immédiatement	Non, il faut re-planifier explicitement
Coût en tokens	Plus élevé	Réduit (#token ReAct / 5 sur HotpotQA)
Performances	Meilleures que CoT et actions individuellement	Légèrement supérieures à ReAct (+4% sur HotpotQA)

Mémoire : au-delà de la fenêtre de contexte

La fenêtre de contexte (séance 4) est une mémoire de **travail** : tout est oublié à la fin de la session, et sa taille est limitée.

Pour un agent qui doit fonctionner sur la durée (plusieurs sessions, des jours d'affilée), il faut une mémoire **persistante**, qui survive au-delà d'un seul appel au modèle.

Deux approches complémentaires : gérer la mémoire comme un **système d'exploitation** (MemGPT), ou comme un **journal d'expériences** consultable (Generative Agents).

MemGPT : le LLM comme système d'exploitation

Packer et al. (2023, UC Berkeley) dans *"MemGPT: Towards LLMs as Operating Systems"*.

Gestion virtuelle du contexte = une technique qui s'inspire des systèmes de mémoire hiérarchique des systèmes d'exploitation (OS) traditionnels, lesquels donnent l'illusion de ressources mémoire importantes en déplaçant les données entre mémoire rapide et mémoire lente.

Analogie directe avec la mémoire virtuelle :

Système d'exploitation	MemGPT
RAM (rapide, limitée)	Contexte principal du LLM
Disque (lent, immense)	Mémoire externe (archive)
Pagination (swap)	Le LLM **lui-même** décide quoi charger/décharger, via des appels de fonction

MemGPT : le LLM comme système d'exploitation

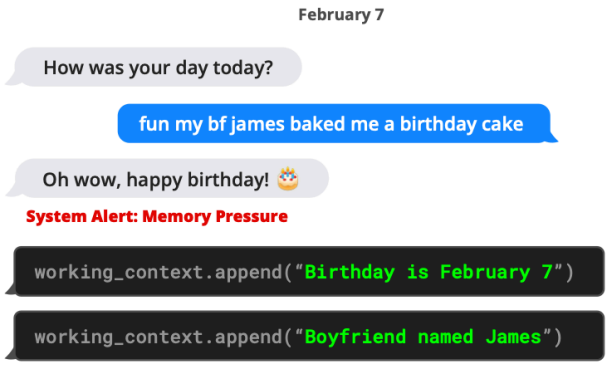


Figure 1. MemGPT (left) writes data to persistent memory after it receives a system alert about limited context space.

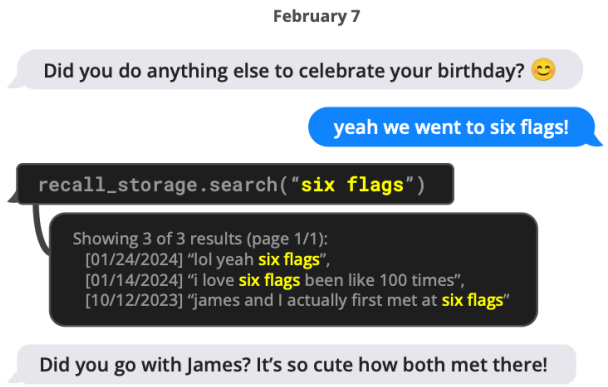


Figure 2. MemGPT (left) can search out-of-context data to bring relevant information into the current context window.

Le modèle peut explicitement appeler `archive_memory_search()` ou `core_memory_append()` pour gérer sa propre mémoire — la gestion de la mémoire devient une **compétence apprise**, pas juste un mécanisme externe.

Performances accrues pour l'analyse de documents et des chats multi-sessions.

Generative Agents

Park et al. (2023, Stanford/Google) dans *"Generative Agents: Interactive Simulacra of Human Behavior"*.
25 agents LLM vivent dans un village simulé ("Smallville").



Generative Agents : un journal d'expériences

Chaque agent a un **flux de mémoire** (*memory stream*) : chaque observation/action est stockée en langage naturel, horodatée.

Pour agir, un agent **récupère** les souvenirs pertinents selon 3 critères combinés :

```
score = récence + importance + pertinence (similarité avec la situation actuelle)
```

Périodiquement, l'agent **réfléchit** (*reflection*) : il relit ses souvenirs récents et en tire des observations de plus haut niveau (ex. "je remarque que je suis souvent interrompu le matin"), elles-mêmes stockées comme nouveaux souvenirs.

Generative Agents : un journal d'expériences

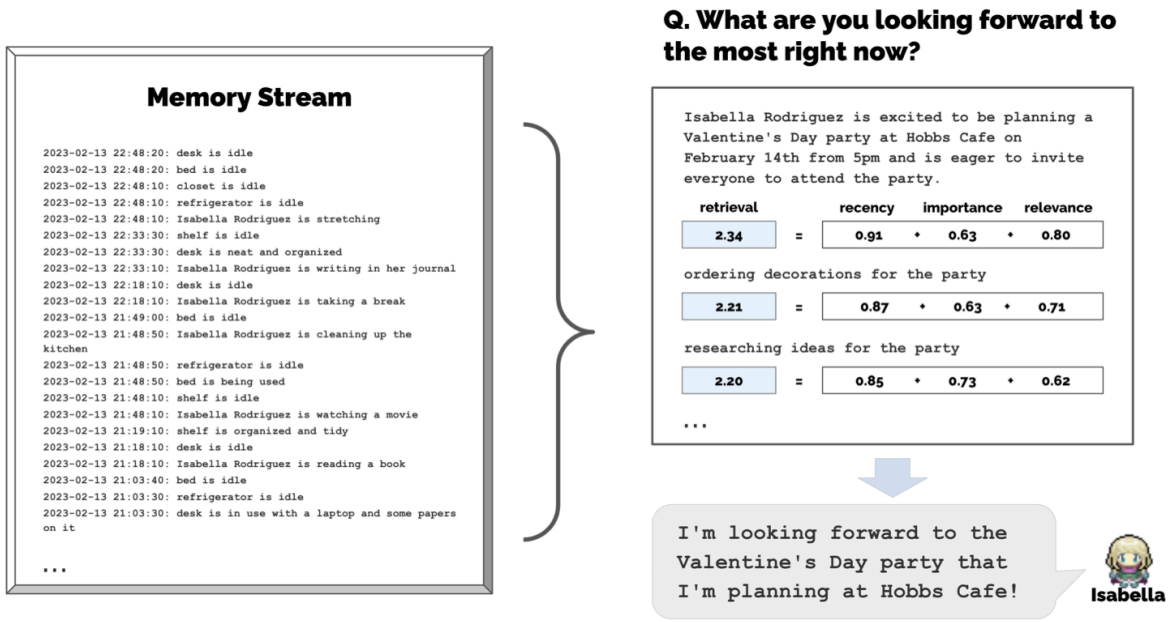


Figure 6: The memory stream comprises a large number of observations that are relevant and irrelevant to the agent's current situation. Retrieval identifies a subset of these observations that should be passed to the language model to condition its response to the situation.

Systèmes multi-agents

Plutôt qu'un seul LLM qui fait tout, on peut faire collaborer **plusieurs instances** spécialisées.

- **Décomposition de tâches** : un agent "chef d'orchestre" délègue des sous-tâches à des agents spécialisés (ex. un agent qui lit du code, un autre qui écrit des tests), c'est le principe des *subagents* dans des outils comme Claude Code
- **Vérification croisée** : plusieurs agents indépendants peuvent se corriger mutuellement

Du et al. (2023, MIT/Google) dans *"Improving Factuality and Reasoning through Multiagent Debate"* : plusieurs instances du même LLM répondent **indépendamment** à une question, puis voient les réponses des autres et les critiquent, sur plusieurs tours → la réponse finale (majoritaire après débat) est **plus factuelle** qu'une réponse simple ou même qu'un simple vote majoritaire sans débat.

🧩 Quiz — Architectures agentiques

1. Quelle est la différence structurelle entre ReAct et ReWOO dans la façon de traiter un plan d'action ?

2. Dans MemGPT, qui décide quand faire passer une information de la mémoire principale vers la mémoire archivée ?

🧩 Quiz — Réponses

1. ReAct alterne raisonnement/action/observation **à chaque étape** (adaptatif, mais coûteux) ; ReWOO génère **tout le plan d'un coup**, l'exécute, puis conclut (moins cher, moins réactif à une surprise).

2. Le **LLM lui-même**, via des appels de fonction explicites (`archive_memory_search` , `core_memory_append`) — la gestion de la mémoire est une action que le modèle apprend à déclencher, pas un mécanisme purement externe.

2. Modèles de raisonnement

Une nouvelle direction : le calcul au moment de l'inférence

Séance 3 (Chinchilla) : à budget de calcul fixé, on choisit la taille du modèle et la quantité de données **d'entraînement**.

Nouvelle question : et si, au lieu d'entraîner un modèle plus gros, on laissait le modèle **plus de temps et générer plus de token** au moment de répondre ?

```
Scaling classique (séance 3)  : + de paramètres / + de données d'entraînement
Inference-time compute (2024+) : + de calcul au moment de générer la réponse
```

Ce n'est **pas** juste du prompting (Chain-of-Thought, séance 4) : ici, le modèle est **entraîné** pour apprendre à utiliser ce temps et ces tokens de réflexion efficacement.

o1 (OpenAI, 2024)

OpenAI entraîne le modèle, par **renforcement**, à produire une longue séquence de raisonnement interne (*"reasoning tokens"*) **avant** de donner sa réponse finale, un peu comme un Chain-of-Thought, mais appris et optimisé plutôt que simplement demandé par prompt.

Deux leviers de calcul, bien distincts :

	Ce qui change	Effet
Calcul à l'entraînement (RL)	Les **poids** du modèle, une fois pour toutes	Le modèle apprend en général à mieux structurer un raisonnement
Calcul à l'inférence	**Rien** dans les poids — juste le nombre de "reasoning tokens" générés pour **cette requête précise**	Une meilleure réponse à **ce** problème, sans rendre le modèle meilleur sur les suivants

Les deux sont **indépendants** : un modèle bien entraîné mais forcé à répondre immédiatement perd une partie de son potentiel ; à l'inverse, laisser "réfléchir" longtemps un modèle qui n'a jamais appris à structurer son raisonnement n'aide pas, il lui manque la compétence de base, pas le temps.

o1 : un exemple concret

Sur un problème de mathématiques difficile, un modèle "classique" (type GPT-4) répond souvent **directement**, en une seule tentative et se trompe si la première approche est mauvaise.

o1 génère d'abord un raisonnement interne **long** (non montré à l'utilisateur par défaut), qui peut ressembler à :

```
"Essayons par substitution... non, ça bloque sur cette étape.
 Essayons plutôt une approche géométrique... ça marche, je vérifie :
 en remplaçant x=3, l'équation est bien satisfaite. Je peux conclure."

Réponse finale : x = 3
```

Ce raisonnement interne consomme **beaucoup plus de tokens** (donc plus de temps et de coût) qu'une réponse directe, d'où les gains sur les tâches de raisonnement (maths, code, sciences), au prix d'une latence et d'un coût par requête bien plus élevés.

o3 et la course au raisonnement

OpenAI **o3** (avril 2025) pousse encore plus loin le même principe : allouer davantage de calcul au moment de l'inférence sur les problèmes qui le nécessitent.

Résultats marquants sur des benchmarks conçus pour résister à la mémorisation (ex. ARC-AGI, problèmes de raisonnement abstrait inédits) — mais à un coût par requête qui peut devenir **très élevé** sur les problèmes les plus difficiles, car le modèle peut "réfléchir" pendant très longtemps.

- Le compromis coût/performance devient un **paramètre de décision explicite** : on choisit combien de calcul allouer à l'inférence selon la difficulté perçue de la tâche.

DeepSeek-R1 : contexte

DeepSeek-AI (2025) dans *"DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning"*.

Premier modèle de raisonnement **open-weight** à rivaliser avec o1 sur les benchmarks de maths/code, entraîné pour une fraction du coût annoncé des modèles propriétaires équivalents, d'où l'onde de choc dans l'industrie en janvier 2025 (le "moment DeepSeek").

Point de départ : leur propre modèle de base déjà pré-entraîné, **DeepSeek-V3-Base** (séance 3 : un pré-entraînement classique, avant toute étape de RL).

DeepSeek-R1-Zero : le raisonnement par RL pur

Choix radical : sauter complètement le SFT (séance 3) et appliquer du **RL à grande échelle directement sur le modèle de base**.

```
DeepSeek-V3-Base (modèle de base, pas encore instruction-tuné)
        ↓  RL à grande échelle (GRPO), SANS SFT préalable
DeepSeek-R1-Zero
```

À chaque exemple d'entraînement (un problème de maths ou de code avec une solution vérifiable), le modèle génère une réponse, reçoit une **récompense automatique** (détail au slide suivant), et ses poids sont ajustés pour rendre plus probables les réponses bien récompensées.

Aucun humain n'a jamais montré au modèle **comment** raisonner étape par étape — seul le signal "bonne ou mauvaise réponse finale", répété des milliers de fois, est utilisé.

Comment la récompense est-elle définie ?

Différence essentielle avec le RLHF classique (séance 3) : **pas de modèle de récompense appris**. La récompense est **calculée par des règles fixes, programmées à l'avance**, pas par un réseau de neurones entraîné sur des préférences humaines.

```
Récompense de justesse (accuracy reward) :
  - Maths : la réponse finale extraite correspond-elle exactement
            à la solution connue ? (comparaison de chaînes/valeurs)
  - Code  : le code généré compile-t-il ET passe-t-il les tests
            unitaires fournis ? (exécution réelle, résultat binaire)

Récompense de format (format reward) :
  - Le raisonnement est-il bien encadré par des balises <think>...</think>
    avant la réponse finale ?
```

Un modèle de récompense **appris** peut être "trompé" (*reward hacking*) ; une règle de vérification automatique (le code s'exécute-t-il ? la réponse est-elle juste ?) ne peut pas être trompée de la même façon, mais cela ne fonctionne que pour des domaines **vérifiables** (maths, code), pas pour une réponse ouverte/créative.

Le raisonnement et ses limites

Résultat surprenant : sans aucun exemple humain de raisonnement, R1-Zero développe spontanément un raisonnement **long**, avec de l'**auto-vérification**, et même un "**aha moment**" documenté où le modèle écrit littéralement *"Wait, let me re-check this"* ou *"That's an aha moment I can flag here"* en plein raisonnement.

Mais R1-Zero a aussi des défauts bien réels :

- **Mélange de langues** dans une même réponse (anglais/chinois entremêlés)
- Raisonnement parfois **peu lisible** pour un humain

→ Optimiser uniquement la justesse de la réponse finale ne garantit **pas** que le chemin pour y arriver soit compréhensible.

DeepSeek-R1 : le pipeline complet

Pour corriger ces défauts, DeepSeek-R1 (version finale) ajoute plusieurs étapes autour du RL pur de R1-Zero :

```
1. Cold-start SFT   : un petit jeu de données de raisonnements longs,
                      bien formatés et lisibles → fine-tuning initial léger
2. RL "raisonnement" : même RL que R1-Zero (GRPO + récompenses de justesse
                      et de format), + une récompense de cohérence de langue
3. Rejection sampling + SFT : le modèle RL génère de nombreuses réponses ;
                      on filtre les meilleures (règles + un modèle qui juge
                      la qualité) et on ré-entraîne en SFT sur ce mélange
                      (raisonnement + tâches générales : écriture, Q&A...)
4. RL final          : un tour de RL supplémentaire combinant justesse
                      ET préférences humaines (aide, sécurité — comme le
                      RLHF classique de la séance 3), pour l'usage général
```

Distillation

Distillation : DeepSeek-R1 (le grand modèle "professeur") **génère** ~800k exemples de raisonnements longs. Ces exemples servent ensuite à faire un **SFT classique** sur des modèles de base **existants** et plus petits (Qwen2.5, LLaMA 3 — de 1,5B à 70B de paramètres) : on ne change pas leur architecture ni leur pré-entraînement, on les fine-tune juste sur les traces de raisonnement générées par R1. Ces versions distillées sont elles aussi publiées en open-weight, avec de bonnes capacités de raisonnement malgré leur taille réduite.

GRPO en un mot

Les étapes de RL de DeepSeek-R1 utilisent **GRPO** (*Group Relative Policy Optimization*, introduit dans DeepSeekMath, Shao et al. 2024) plutôt que PPO (séance 3, RLHF).

Différence avec PPO : PPO entraîne un modèle de récompense **et** un modèle de valeur (*critic*) séparé pour estimer la récompense attendue. GRPO élimine le critic :

```
1. Pour un même prompt, échantillonner un GROUPE de réponses
2. Calculer la récompense de chacune (règles du slide précédent :
   accuracy reward + format reward)
3. Utiliser la MOYENNE du groupe comme référence
4. Récompenser les réponses au-dessus de la moyenne du groupe, pénaliser celles en-dessous
```

Plus simple et moins coûteux à entraîner que PPO (pas de second réseau à maintenir) — un facteur qui explique en partie le coût d'entraînement réduit de DeepSeek-R1 par rapport aux modèles de raisonnement propriétaires.

Pourquoi le calcul à l'inférence fonctionne-t-il ?

Snell et al. (2024, Google DeepMind/UC Berkeley) dans *"Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters"*.

Pour un budget de calcul **fixé**, allouer plus de calcul à l'**inférence** (échantillonner plus, vérifier, chercher) peut battre le fait d'entraîner un modèle plus **gros**, particulièrement sur des problèmes où **vérifier** une réponse est plus facile que la **générer**.

Nuance importante : ça ne marche pas pour tous les problèmes, sur des problèmes très difficiles, aucune quantité de calcul d'inférence ne compense un modèle de base trop faible.

Vérifier plutôt que juste répondre

Lightman et al. (2023, OpenAI) dans *"Let's Verify Step by Step"*.

Comparaison de deux façons de superviser le raisonnement :

	Récompense sur le résultat	Récompense sur chaque étape
Nom	*Outcome Reward Model* (ORM)	*Process Reward Model* (PRM)
Principe	Note seulement la réponse finale	Note chaque étape intermédiaire du raisonnement
Résultat sur MATH	Bon	**Meilleur** — corrige les erreurs de raisonnement même quand la réponse finale est juste par chance

Ces vérificateurs, combinés au *self-consistency* (séance 4) et à la recherche dans l'espace des raisonnements (Tree of Thoughts), forment la boîte à outils du **test-time compute**.

🧩 Quiz — Modèles de raisonnement

1. Quelle est la différence entre le scaling "à l'entraînement" (Chinchilla, séance 3) et le scaling "à l'inférence" (o1, o3) ?

2. Pourquoi le résultat de DeepSeek-R1-Zero est-il surprenant du point de vue de l'entraînement des LLMs ?

3. En quoi la récompense utilisée pour entraîner DeepSeek-R1-Zero est-elle différente du modèle de récompense du RLHF classique (séance 3) ?

4. Quelle est la principale différence entre un Outcome Reward Model et un Process Reward Model ?

🧩 Quiz — Réponses

1. Le scaling à l'entraînement augmente la taille du modèle ou des données **une fois pour toutes** ; le scaling à l'inférence dépense plus de calcul **à chaque requête**, en laissant le modèle "réfléchir" plus longtemps sur ce problème précis.

2. Un comportement de raisonnement complexe (auto-vérification, "aha moment") **émerge uniquement de la récompense RL**, sans aucun exemple humain de raisonnement (pas de SFT préalable) — contredisant l'idée que ces comportements doivent être démontrés explicitement.

3. Le RLHF utilise un **modèle de récompense appris** (un réseau de neurones entraîné sur des comparaisons humaines) ; DeepSeek-R1-Zero utilise une récompense **basée sur des règles fixes** (la réponse maths est-elle exacte ? le code passe-t-il les tests ?), non apprise et donc plus difficile à "tromper" (reward hacking).

4. L'ORM ne note que la réponse finale ; le PRM note **chaque étape** du raisonnement, ce qui permet de détecter des erreurs de raisonnement même quand la réponse finale est correcte par coïncidence.

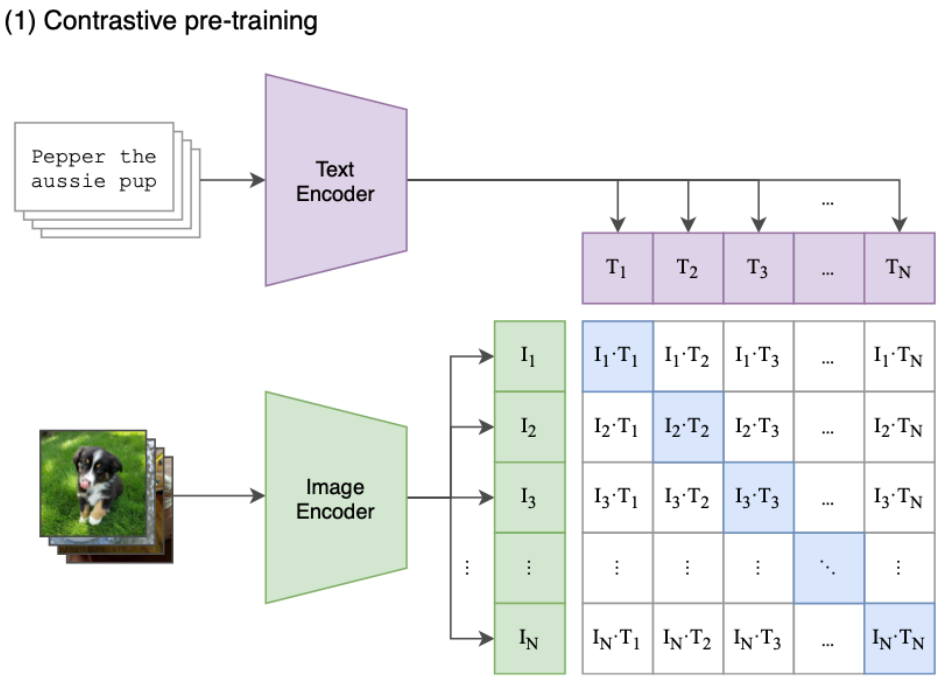
3. (bonus) Multimodalité : vision-langage en bref

Pourquoi le texte seul ne suffit pas

Beaucoup de tâches réelles nécessitent de **percevoir une image** : lire un graphique, décrire une capture d'écran, analyser un document scanné, comprendre un diagramme d'architecture.

Idée centrale : convertir l'image en une séquence de "tokens visuels", pour que le **même** Transformer qui traite le texte puisse aussi traiter l'image, avec le **même mécanisme d'attention**.

CLIP : aligner texte et image



CLIP : aligner texte et image

Radford et al. (2021, OpenAI) dans *"Learning Transferable Visual Models From Natural Language Supervision"*.

Entraîné par **apprentissage contrastif** sur ~400 millions de paires (image, légende) collectées sur le web : rapprocher l'embedding d'une image de celui de **sa** légende, et l'éloigner des légendes des autres images du batch.

```
image(chat) ~ texte("une photo d'un chat")     ← rapprochés
image(chat) ≠ texte("une photo d'un avion")    ← éloignés
```

Résultat : un espace d'embeddings **partagé** entre texte et image, permettant de la classification **zero-shot** (comparer une image à plusieurs labels textuels candidats, sans entraînement spécifique).

Brancher la vision sur un LLM : LLaVA

Liu et al. (2023) — *"Visual Instruction Tuning"*.

```
Image → découpée en patchs → Vision Transformer (ViT)  → embeddings visuels
                                                                ↓
                                                  petite couche de projection (entraînée)
                                                                ↓
                                              "tokens visuels" injectés dans le LLM,
                                              mélangés aux tokens de texte, même attention
```

Recette légère : encodeur visuel (CLIP) **gelé** + LLM (Vicuna) + une **petite** couche de projection entraînée sur des instructions visuelles générées par GPT-4 — pas besoin de ré-entraîner tout le modèle de langage.

Modèles multimodaux natifs

Les modèles de production récents (GPT-4o, Claude 3+, Gemini) ne "branchent" plus la vision après coup : ils sont **entraînés conjointement** sur du texte et de l'image **entrelacés** dès le pré-entraînement.

- Meilleure intégration entre les deux modalités qu'une architecture "encodeur gelé + adaptateur" (LLaVA)
- Permet aussi, pour certains modèles, de **générer** des images ou de l'audio, pas seulement de les comprendre
- Le principe reste le même que pour le texte (séance 3) : plus de données multimodales, plus de calcul → meilleures capacités

🧩 Quiz — Multimodalité

1. Comment un modèle vision-langage type LLaVA "voit"-il une image, techniquement ?

2. Qu'est-ce que CLIP apprend exactement, et sur quel type de données ?

🧩 Quiz — Réponses

1. L'image est découpée en patchs, encodée par un Vision Transformer, puis projetée dans le même espace d'embeddings que les tokens de texte — le LLM leur applique ensuite la **même** attention qu'à des tokens textuels.

2. Un espace d'embeddings **partagé** entre texte et image, en rapprochant chaque image de sa légende (et en l'éloignant des autres légendes du batch) sur ~400 millions de paires (image, texte) du web.

4. Préparation à l'examen

Vue d'ensemble du cours

```
Séance 1 : Bases du NLP (tokens, stemming/lemmatisation, stop words,
           loi de Zipf) et rappels ML (loss, gradient, overfitting)
    ↓
Séance 2 : Embeddings (Word2Vec, GloVe), RNNs, Transformer & attention
    ↓
Séance 3 : Pré-entraînement à grande échelle, Chinchilla,
           Instruction tuning, RLHF/DPO, Évaluation
    ↓
Séance 4 : Prompting, contraintes d'inférence, RAG, function calling
    ↓
Séance 5 : Agents (ReAct, planification, mémoire), modèles de
           raisonnement (o1/o3, DeepSeek-R1), multimodalité
```

Un seul fil conducteur : les bases du traitement automatique du texte et de l'entraînement des modèles deep learning (séance 1), représentation du texte en vecteurs et architecture transformer (séance 2), comment un LLM est entraîné (séance 3), puis comment il est utilisé et étendu à l'appel et utilisateur d'outils externes (séance 4), enfin comment orchestrer la réalisation de tâches avec un ou plusieurs LLMs (séances 5).

🧩 Quiz de révision intéractif

https://app.wooclap.com/MUZAQOT?from=instruction-slide



🧩 Quiz de révision — Séances 1 & 2 (et un peu 3)

1. *(séance 1)* Quelle est la différence entre le stemming et la lemmatisation ?

2. *(séance 1)* Un modèle atteint 98% d'accuracy en entraînement et 55% en test. Quel est le diagnostic, et citez une solution.

3. *(séance 2)* Pourquoi un Transformer s'entraîne-t-il plus vite qu'un RNN sur GPU ?

4. *(séance 2)* Pourquoi les embeddings Word2Vec ne peuvent-ils pas distinguer les deux sens du mot "batterie" (instrument vs pile), contrairement aux embeddings d'un Transformer ?

5. *(séance 3)* Selon les lois de Chinchilla, que faut-il faire croître **en même temps** que la taille d'un modèle pour utiliser un budget de calcul de façon optimale ?

🧩 Quiz de révision — Séances 1 & 2 (et un peu 3) : Réponses

1. Le **stemming** tronque brutalement un mot selon des règles fixes (`"courais"` → `"cour"`) ; la **lemmatisation** trouve sa forme canonique en tenant compte du contexte grammatical (`"courais"` → `"courir"`).

2. C'est du **surapprentissage** (*overfitting*) : le modèle mémorise les données d'entraînement au lieu d'apprendre des patterns généraux. Solutions : dropout, weight decay, early stopping, ou plus de données.

3. Parce que l'attention calcule les relations entre tous les tokens **en parallèle** via des multiplications matricielles, alors qu'un RNN traite les tokens **séquentiellement**, un par un.

4. Word2Vec attribue **un seul vecteur fixe** par mot, indépendamment du contexte ; un Transformer produit des embeddings **contextuels**, qui changent selon les mots environnants.

5. La **quantité de données d'entraînement** (le nombre de tokens) — règle empirique : environ 20 tokens par paramètre.

🧩 Quiz de révision — Séances 3, 4 & 5

1. *(séance 3)* Quelle est la différence d'objectif entre le SFT et le RLHF/DPO ?

2. *(séance 3)* Pourquoi un score élevé sur MMLU ne garantit-il pas qu'un modèle est réellement meilleur ?

3. *(séance 4)* Pourquoi doubler la fenêtre de contexte d'un Transformer coûte-t-il plus que deux fois plus cher en calcul ?

4. *(séance 4)* Quelle est la principale limite d'un LLM que le RAG cherche à résoudre ?

5. *(séance 5)* Quel est l'avantage de ReAct par rapport à un simple prompt Chain-of-Thought pour répondre à une question nécessitant une information récente (ex. la météo actuelle) ?

🧩 Quiz de révision — Séances 3, 4 & 5 : Réponses

1. Le **SFT** apprend au modèle à suivre le **format** instruction → réponse (un exemple correct parmi d'autres) ; le **RLHF/DPO** apprend une **préférence relative** entre plusieurs réponses possibles, pour affiner le comportement au-delà du simple format.

2. Risque de **contamination** : le benchmark (ou ses réponses) peut avoir fuité dans les données de pré-entraînement, gonflant le score sans réelle généralisation.

3. L'attention calcule une interaction entre **chaque paire de tokens** : le coût est **quadratique** ($O(n^2)$) en la longueur de séquence, pas linéaire.

4. Les connaissances d'un LLM sont **figées** à sa date de coupure d'entraînement, et n'incluent aucune donnée privée/interne — le RAG permet d'injecter des informations à jour ou privées **sans ré-entraîner** le modèle.

5. Un simple CoT raisonne uniquement à partir de ce que le modèle **sait déjà** (figé à l'entraînement) ; ReAct peut **agir** (ex. lancer une recherche) pour obtenir une information réelle et à jour, puis raisonner à partir du résultat observé.

Modalités de l'examen

QCM 40 questions - 1 heure - Vendredi 31 juillet

En attendant, la meilleure préparation reste de **refaire les quiz** de chaque séance et de savoir **expliquer avec vos mots** (pas juste réciter) chaque étape du pipeline : tokenization → pré-entraînement → alignement → évaluation → usage (prompting, RAG, outils, agents).

Questions ?

Merci pour votre attention pendant ce semestre !

`celia.nouri@inria.fr`

Ressources

- **ReAct** : Yao et al. (2022/2023); arxiv.org/abs/2210.03629
- **ReWOO** : Xu et al. (2023); arxiv.org/abs/2305.18323
- **MemGPT** : Packer et al. (2023); arxiv.org/abs/2310.08560
- **Generative Agents** : Park et al. (2023); arxiv.org/abs/2304.03442
- **Multiagent Debate** : Du et al. (2023); arxiv.org/abs/2305.14325
- **DeepSeek-R1** : DeepSeek-AI (2025); arxiv.org/abs/2501.12948
- **DeepSeekMath (GRPO)** : Shao et al. (2024); arxiv.org/abs/2402.03300
- **Scaling Test-Time Compute** : Snell et al. (2024); arxiv.org/abs/2408.03314
- **Let's Verify Step by Step** : Lightman et al. (2023); arxiv.org/abs/2305.20050
- **CLIP** : Radford et al. (2021); arxiv.org/abs/2103.00020
- **LLaVA** : Liu et al. (2023); arxiv.org/abs/2304.08485
- **Vision Transformer (ViT)** : Dosovitskiy et al. (2020); arxiv.org/abs/2010.11929